Lecture 6: Reinforcement Learning in Finance

Chao ZHANG

March 18, 2025

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

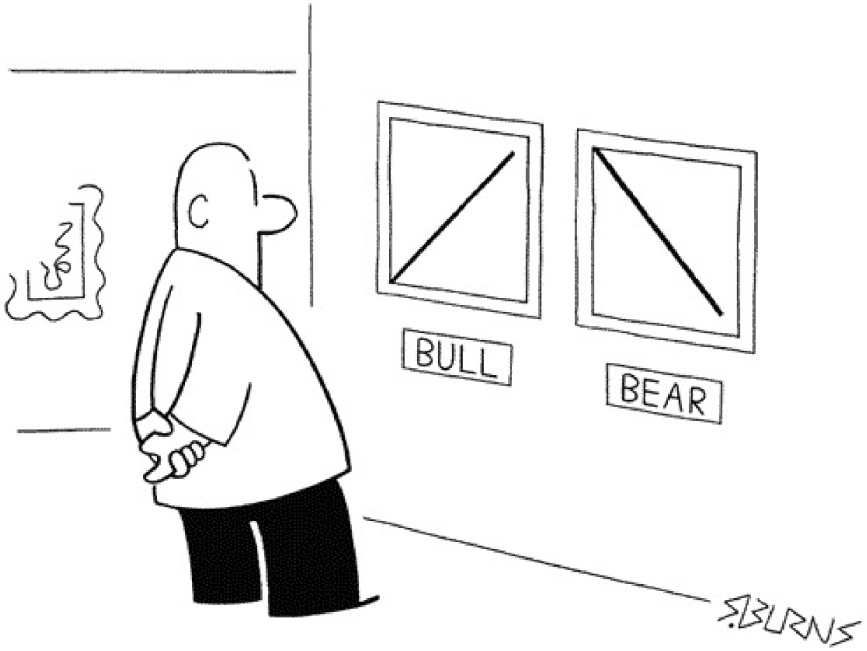
Optimal Liquidation: Motivation

A Basic Question:

How to optimally liquidate (sell) a portfolio with X shares over a given time period $[0, T]$?

Classic Trade-off

- Liquidating fast is costly: transaction costs and market impact.
- But if one liquidates too slowly...



Optimal Liquidation: Motivation

... the price may go down while we are liquidating...

... and we would have been better executing faster.

$\Longrightarrow$ **Need to find an optimal trading schedule to trade-off:**

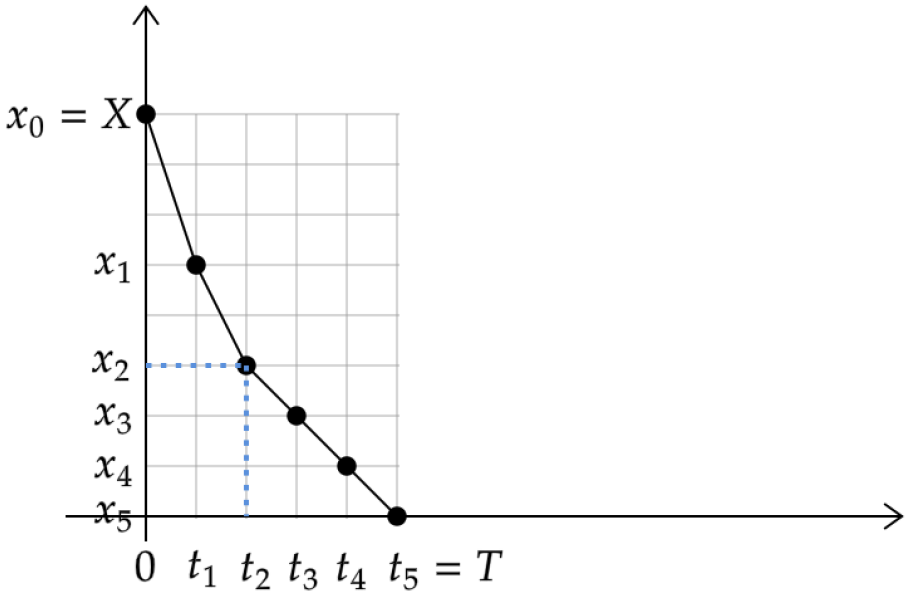
- Trading cost (transaction cost and market impact)
- Future uncertainty (distant future is harder to predict than recent future)

Almgren-Chriss Framework

- First framework to address this trade-off from a mathematical point of view
- Proposed by Rob Almgren and Neil Chriss in Almgren and Chriss [2001] – Almgren-Chriss Framework
- Discussed both the market impact and execution cost
- For both single-asset liquidation and portfolio liquidation

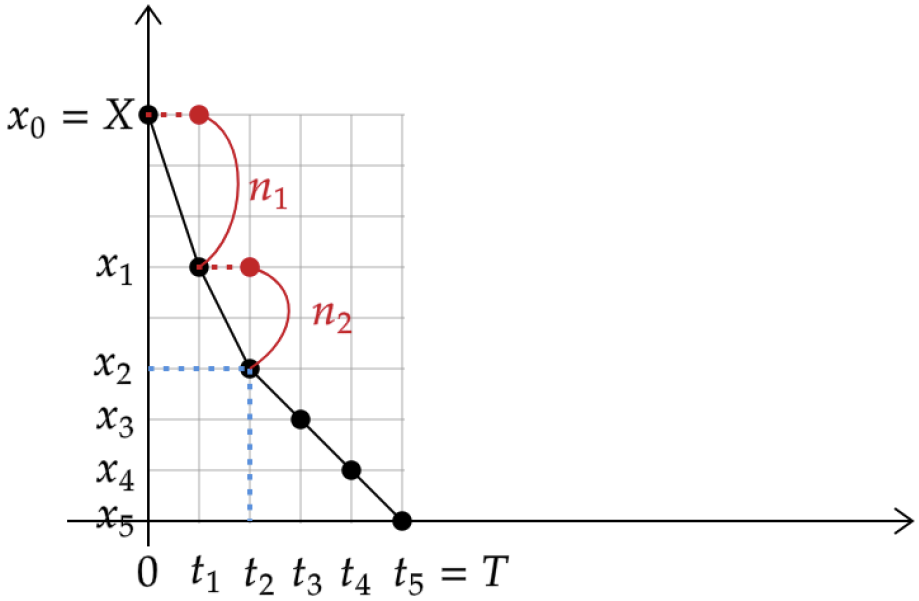
Almgren-Chriss Framework

- A trader holds a block of X shares of an asset at time $t_0 = 0$ and aim to liquidate all shares before (or up to) time $T > 0$
- The duration $[0, T]$ is divided into N interval of equal length $\tau = \frac{T}{N}$ and the agent can trade at each time $t_k := k\tau$ with $k = 0, 1, \cdots, N$
- **Trading trajectory**: $(x_0, \cdots, x_N)$ where x_k is the number of units that the agent plans to hold at time t_k.



Almgren-Chriss Framework for Single-asset

- Boundary conditions: $x_0 = X$ (number of shares hold at time 0 to be sold); $x_N = 0$ (total liquidation at time N)
- Trading list: $(n_1, ..., n_N)$ where $n_k = x_{k-1} - x_k$ is the number of shares sold between t_{k-1} and t_k
- Relationship: $x_k = X - \sum_{j=1}^{k} n_j = \sum_{j=k+1}^{N} n_j$



Price Dynamics for Single-asset

- Price of the asset at time 0 is S_0
- Without liquidation decisions, the base (unaffected) price follows a **random walk**:

$$S_k = S_{k-1} + \sigma \tau^{1/2} \xi_k \tag{1}$$

  - $(\xi_k)_{1 \leq k \leq N}$ is a family of i.i.d. noise with zero mean and unit variance
  - $\sigma > 0$ is the volatility of the asset
- $$dS_t = \sigma dW_t$$

  - W_t is a standard Brownian motion.

Temporary Market Impact

- A short-term price deviation
- the immediate effect of an order absorbing liquidity from the order book.

$$P_k = S_k - h(\frac{n_k}{\tau}), \tag{2}$$

where h is the temporary impact function and P_k is the actual traded price at time t_k.

- Causes:
  - Slippage due to crossing the bid-ask spread.
  - Order book depletion
- Characteristics:
  - Reversible: Once the order is fully executed and liquidity replenishes, prices may revert.
  - Non-informational: It does not signal any fundamental change in the asset's value.
  - Linked to order book dynamics: Temporary impact depends on order book depth and liquidity resilience.

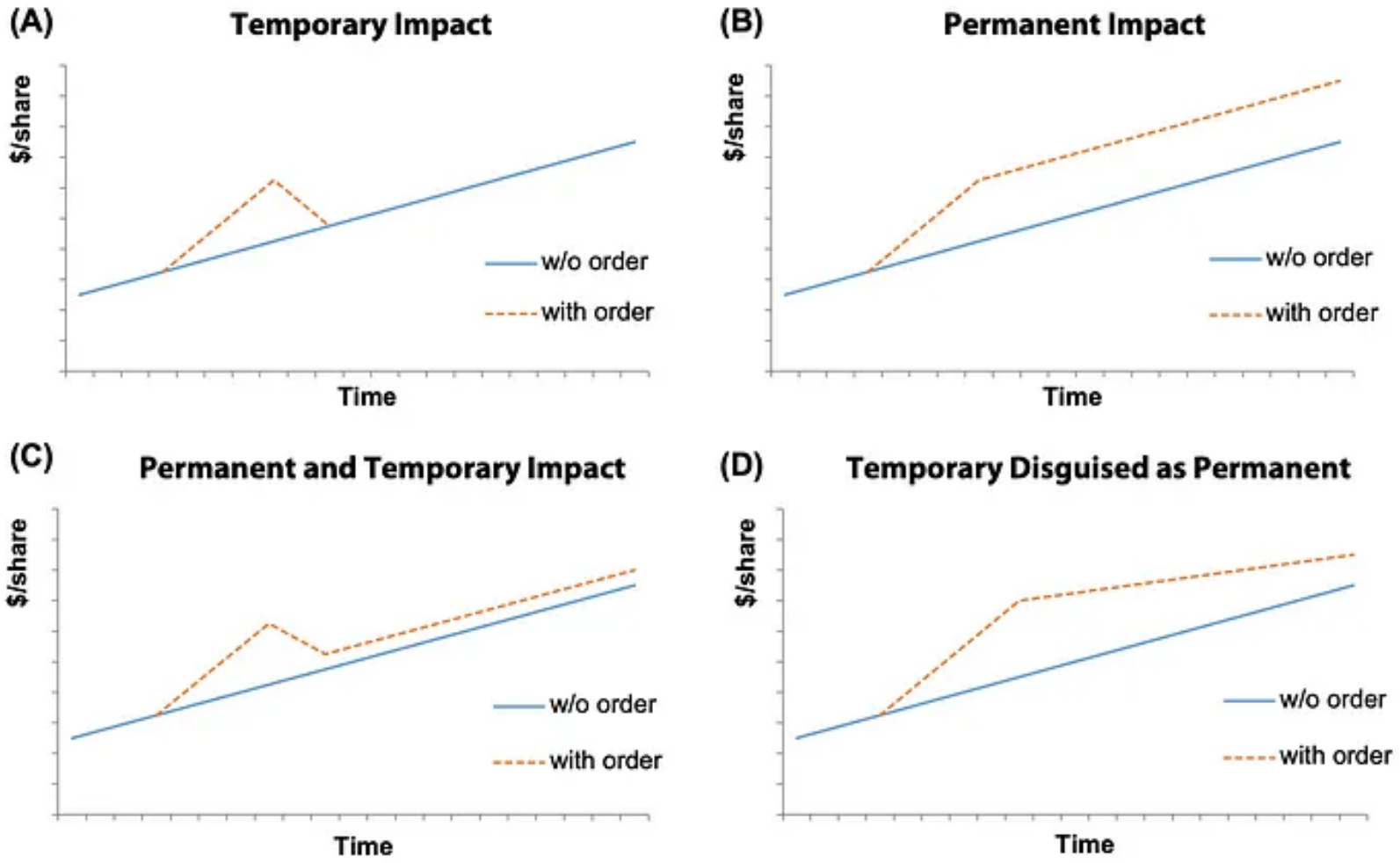
Permanent market impact

- A long-term change
- typically driven by new information inferred by the market

$$S_k = S_{k-1} + \sigma\tau^{1/2}\xi_k - \tau g(\frac{n_k}{\tau}) \tag{3}$$

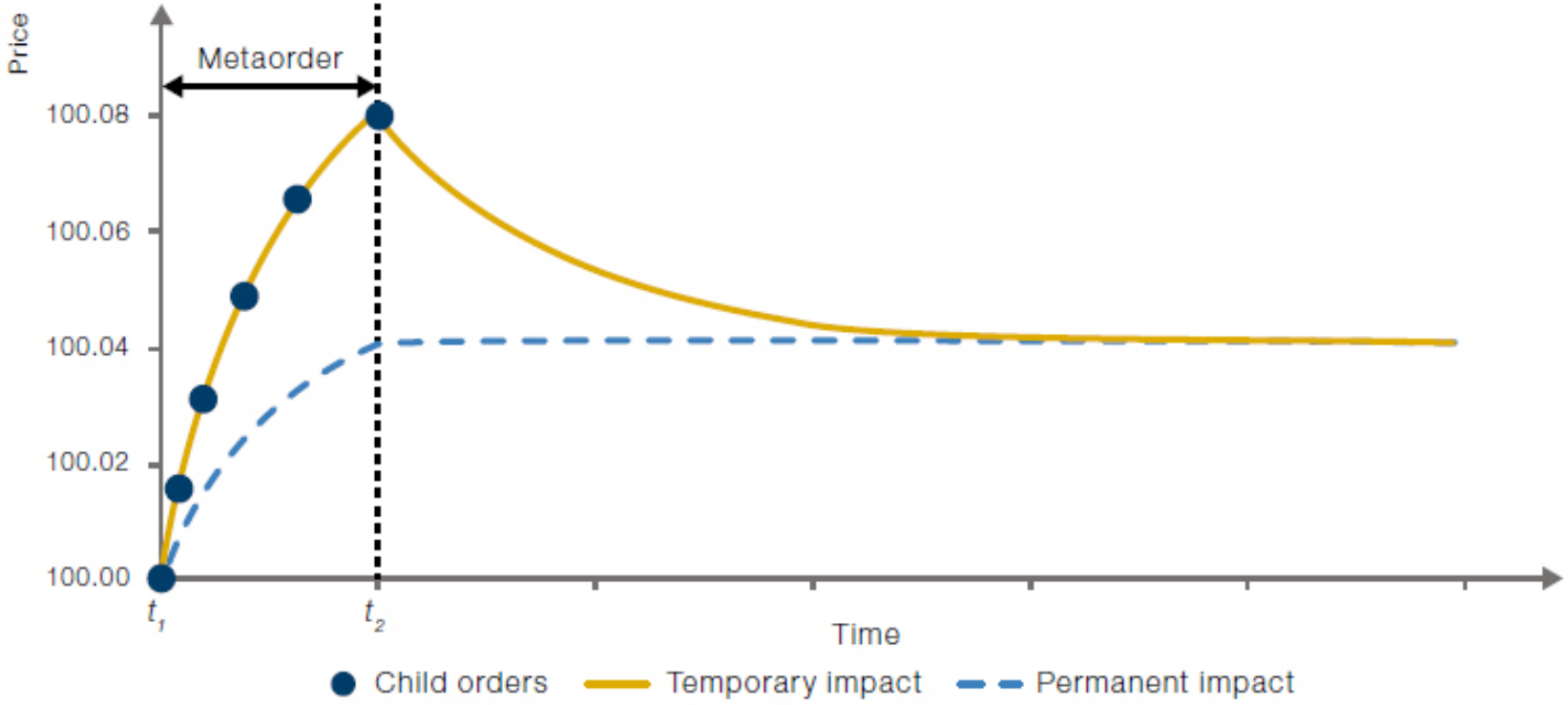
  g is the permanent impact map where $g(v)$ is a function of the average rate of trading $v = n_k/\tau$ during the interval t_{k-1} and t_k
- Causes:
  - Large trades as signals of private information.
  - Structural shifts in supply and demand due to large institutional trades.
- Characteristics:
  - Irreversible (or persistent): Even after the trade is completed, the price does not revert.
  - Informational: If the market believes the order reveals information about fundamental value, prices will adjust permanently.
  - Highly liquid markets tend to exhibit lower permanent impact, while illiquid markets are more sensitive.
- Example: If a hedge fund starts accumulating a stock aggressively, the market may infer that the fund has positive private information, leading to a sustained increase in price.

Market Impact



■ **FIGURE 4.4** Market Impact Trajectories.

Market Impact



Example: Problem Setup

- **Order Size:** $X = 1000$ shares.
- **Time Horizon:** $T = 1$ hour, divided into 6 equal intervals ($\Delta t = \frac{1}{6}$ hour).
- **Equal Trade Size:** $x_i = \frac{1000}{6} \approx 166.67$ shares per interval.
- **Initial Asset Price:** $S_0 = 1000$.
- **Market Impact Parameters:** Linear
  - $\gamma = 0.1$ (Permanent impact coefficient, price reduction per share executed).
  - $\eta = 0.05$ (Temporary impact coefficient).

Computation Process

For each interval i:

- **Base Price:**

$$S_i = S_0 - \gamma \times (\text{cumulative shares executed before interval } i)$$

- **Trading Rate:**

$$v_i = \frac{x_i}{\Delta t} = \frac{166.67}{1/6} = 1000 \quad \text{shares per hour}$$

- **Temporary Impact:**

$$\text{Impact} = \eta \times v_i = 0.05 \times 1000 = 50$$

- **Execution Price:**

$$P_i = S_i - 50$$

- **Revenue per Interval:**

$$R_i = x_i \times P_i$$

Interval Calculations

Interval	Executed	Base Price S_i	Exec. Price P_i	Revenue R_i
1	166.67	1000.0	950.0	$\approx \$158,333$
2	166.67	983.33	933.33	$\approx \$155,556$
3	166.67	966.67	916.67	$\approx \$152,778$
4	166.67	950.0	900.0	$\approx \$150,000$
5	166.67	933.33	883.33	$\approx \$147,222$
6	166.67	916.67	866.67	$\approx \$144,444$
			Total Revenue:	$\approx \$908,333$

- *Base Price Decline:* The base price S_i decreases over time due to cumulative permanent impact.
- *Temporary Impact:* Since the trading rate is constant (1000 shares/hour), the temporary impact (50) remains the same in each interval. $P_i = S_i - 50$ decreases over intervals, leading to reduced revenue per share.

Cost of Trading

The full trading revenue with strategy $(x_0, \cdots, x_N)$ is given by

$$\underbrace{\sum_{k=1}^{N} n_k P_k}_{\text{Full trading revenue}} = \underbrace{XS_0}_{\text{Initial value}} + \underbrace{\sum_{k=1}^{N} \sigma \tau^{1/2} \xi_k x_k}_{\text{volatility effect}} \tag{4}$$

$$- \underbrace{\sum_{k=1}^{N} \tau g(\frac{n_k}{\tau}) x_k}_{\text{loss permanent impact}} - \underbrace{\sum_{k=1}^{N} \tau h(\frac{n_k}{\tau}) n_k}_{\text{loss temporary impact}} \tag{5}$$

Cost of Trading

Implementation shortfall: loss of money due to the implementation of the agent's trading strategy

$$\mathcal{I}_{\mathcal{S}}(x) := XS_0 - \sum_{k=1}^{n} n_k P_k \tag{6}$$

$$= \underbrace{-\sum_{k=1}^{N} \sigma \tau^{1/2} x_k \xi_k}_{\text{vol. cost}} + \underbrace{\sum_{k=1}^{N} \tau g(\frac{n_k}{\tau}) x_k}_{\text{permanent impact cost}} + \underbrace{\sum_{k=1}^{N} \tau h(\frac{n_k}{\tau}) n_k}_{\text{temporary impact cost}} \tag{7}$$

Optimal Liquidation Problem

The trader aims to solve

$$\min_{(x_k)_{0\leq k\leq N}} \mathbb{E}[\mathcal{I}_{\mathcal{S}}(x)] + \lambda \mathrm{Var}[\mathcal{I}_{\mathcal{S}}(x)] \tag{8}$$

where $\lambda \in \mathbb{R}$ is a risk aversion parameter.

- Execution Risk: Spreading the trade over time exposes the trader to price volatility.

$\Longrightarrow$ Explicit solution if g and h are linear.

Optimal Liquidation Problem

Linear case:

- $g(v) = \gamma v$ with some parameter $\gamma > 0$
- $h(v) = \eta v$, with some parameters $\eta > 0$
- with $v = n_k/\tau$ is the trading rate

The optimization problem becomes

$$\mathcal{P}(\lambda) : \min_x U_\lambda(x) \tag{9}$$

with

$$\begin{aligned} U_\lambda(x) &:= \mathbb{E}[\mathcal{I}_\mathcal{S}(x)] + \lambda \mathrm{Var}[\mathcal{I}_\mathcal{S}(x)] \\ &= \frac{1}{2}\gamma X^2 + \frac{\widetilde{\eta}}{\tau}\sum_{k=1}^{N}(x_{k-1} - x_k)^2 + \lambda \sum_{k=1}^{N} \sigma^2 \tau x_k^2, \end{aligned} \tag{10}$$

with $\widetilde{\eta} = \eta - \frac{1}{2}\gamma\tau$.

- Cost Component, Execution Risk

Optimal Liquidation Problem

Two naive strategies:

- Equal sized packets: $n_k = \frac{X}{N}$ so that $x_k = (N-k)\frac{X}{N}$ for any $1 \leq k \leq N$
  - It minimizes $x \longmapsto \mathbb{E}[\mathcal{I}_{\mathcal{S}}(x)]$
  - But high $\mathrm{Var}[\mathcal{I}_{\mathcal{S}}(x)]$
  - $\hookrightarrow$ It corresponds to the case $\lambda = 0$ in $\mathcal{P}_\lambda$
- Eager agent: $n_1 = X, n_2 = \cdots = n_N = 0$, $x_1 = \cdots = x_N = 0$
  - $\mathrm{Var}[\mathcal{I}_{\mathcal{S}}(x)] = 0$
  - But $\mathbb{E}[\mathcal{I}_{\mathcal{S}}(x)]$ can be very large
  - $\hookrightarrow$ It corresponds to a variance minimization only (roughly speaking $\lambda \sim +\infty$ in $\mathcal{P}_\lambda$)

Question: What it the optimal trade-off between these two extrema?

Solve the Optimal Liquidation Problem

- $x \longmapsto U_\lambda(x)$ is quadratic with respect to x and convex as soon as $\widetilde{\eta} = \eta - \frac{1}{2}\gamma\tau > 0$ and $\lambda \geq 0$.
- First order condition gives:

$$\begin{cases} \widetilde{\kappa}^2 x_k = \frac{1}{\tau^2}(x_{k-1} - 2x_k + x_{k+1}), \text{ with } \widetilde{\kappa}^2 = \frac{\lambda\sigma^2}{\widetilde{\eta}} \\ \text{ with } x_0 = X \text{ and } x_N = 0. \end{cases}$$

- The solution to this linear system:

$$\begin{cases} x_k = \frac{\sinh(\kappa(T-t_k))}{\sinh(\kappa T)} X, k = 0, \cdots, N \\ \frac{2}{\tau^2}(\cosh(\kappa\tau) - 1) = \widetilde{\kappa}^2 \\ n_k = \frac{2\sinh(\frac{1}{2}\kappa\tau)}{\sinh(\kappa T)} \cosh(\kappa(T - (k - \frac{1}{2}\tau)))X, \ k = 1, 2, \cdots, N \end{cases}$$

Optimal Strategy (Almgren–Chriss)

- $$\kappa = \sqrt{\frac{0.1 \times 2^2}{0.05}} = \sqrt{8} \approx 2.828.$$
- **Optimal Trajectory:**

$$x(t) = 1000\,\frac{\sinh\big(2.828\,(1-t)\big)}{\sinh(2.828)}.$$

Optimal Strategy: Revenue Calculation

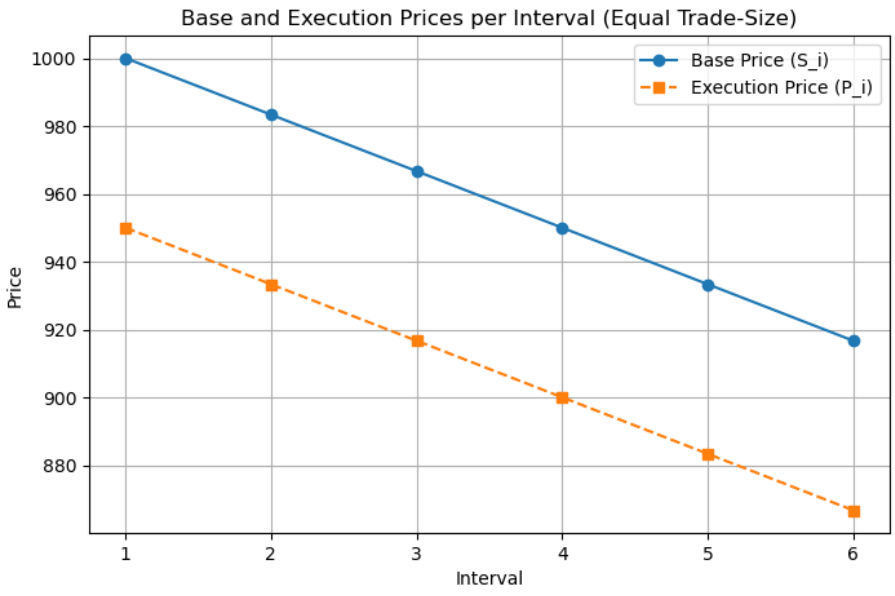
- **Price Dynamics per Interval:**
  - **Base Price:**
    $S_i = S_0 - \gamma\,(\text{cumulative shares executed before interval } i)$.
  - **Temporary Impact:** Trading rate $v_i = \frac{x_i}{\Delta t} = 6\,x_i$; temporary cost is $\eta\,v_i$.
  - **Execution Price:** $P_i = S_i - \eta\,v_i$.
- **Computed Values:**

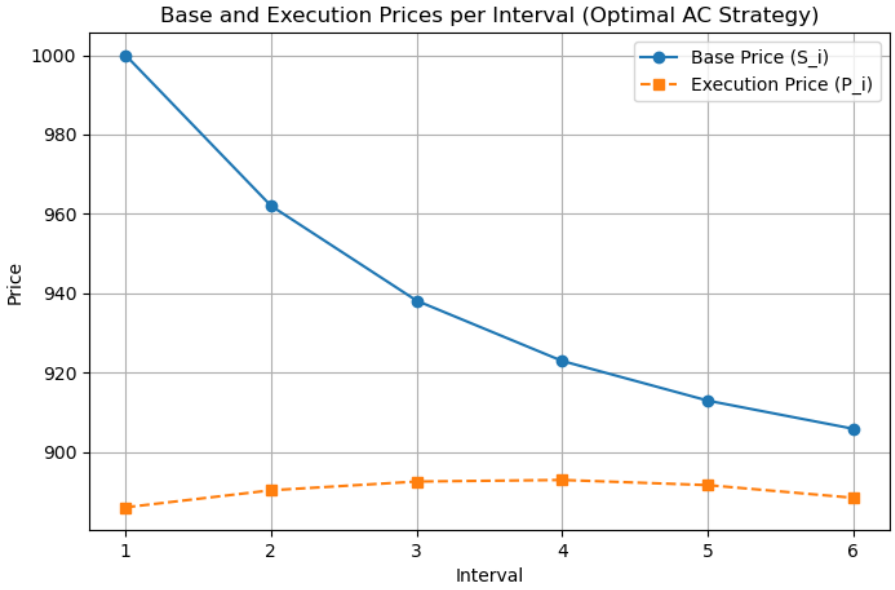
Interval	Shares Executed	Base Price	Exec. Price	Revenue
1	380	1000.0	886.0	\$336,680
2	239	962.0	890.3	\$212,782
3	152	938.1	892.5	\$135,660
4	100	922.9	892.9	\$89,290
5	71	912.9	891.6	\$63,364
6	58	905.8	888.4	\$51,567
			Total Revenue:	\$889,343

Comparison of Strategies

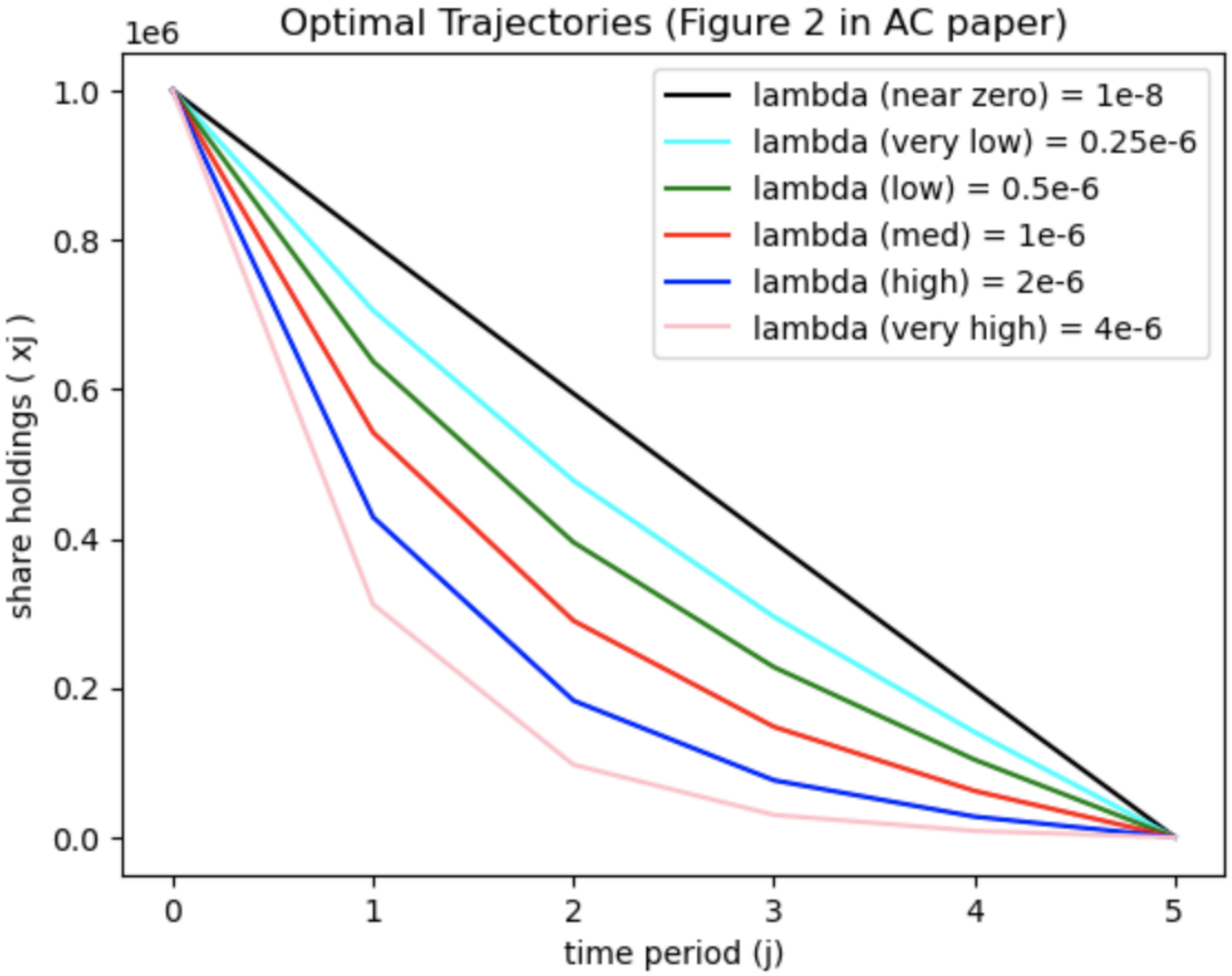
- **Almgren–Chriss (Optimal) Strategy:**
  - **Trading Schedule:** Front-loads execution to reduce risk.
  - **Total Revenue:** $889,343.
  - **Trade-Off:** Lower revenue (due to deeper permanent impact) in exchange for reduced exposure to price risk.
- **Equal-Trade-Size Strategy:**
  - **Trading Schedule:** Uniform execution across intervals.
  - **Total Revenue:** $908,333.
  - **Trade-Off:** Higher revenue per share but potentially higher risk exposure.
- **Key Insight:** The AC strategy minimizes a cost-plus-risk objective; its aggressive early trading lowers risk at the expense of lowering the average effective price.

Prices


Base and Execution Prices per Interval (Equal Trade-Size)
Base Price (S_i)
Execution Price (P_i)
Price
Interval
1000
980
960
940
920
900
880
1
2
3
4
5
6


Base and Execution Prices per Interval (Optimal AC Strategy)
Base Price (S_i)
Execution Price (P_i)
Price
Interval
1000
980
960
940
920
900
1
2
3
4
5
6

Optimal Liquidation Trajectory



Some Practical Questions Related to the Math Framework

1. What price dynamics is S_t modeling?
2. What is the meaning of "market impact"? How to understand it from a practical point of view?
3. How to estimate the model parameters $\lambda, \sigma, \eta, \gamma$ from data?

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

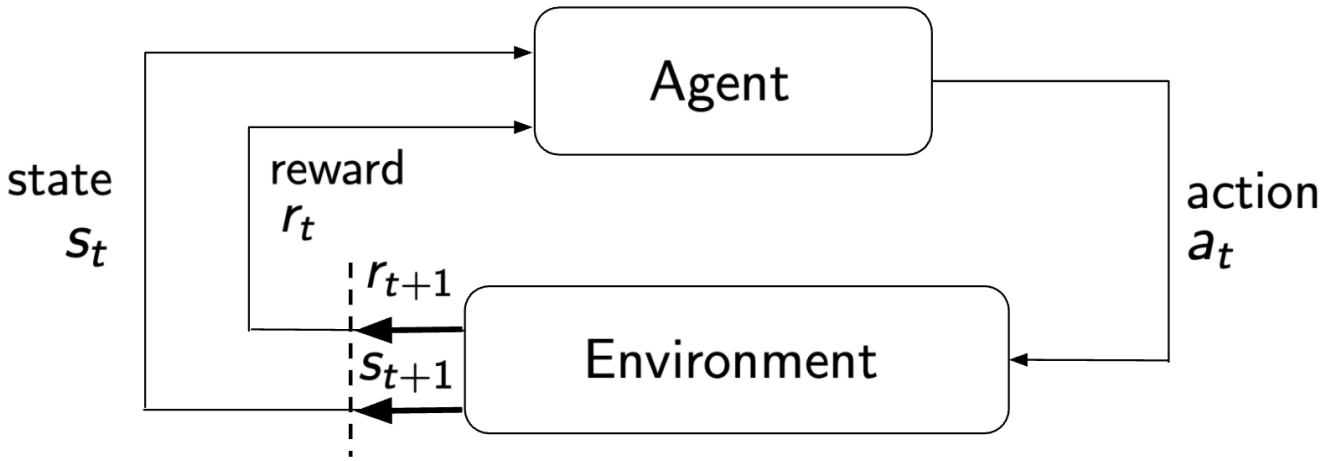
Deep RL

RL in Finance

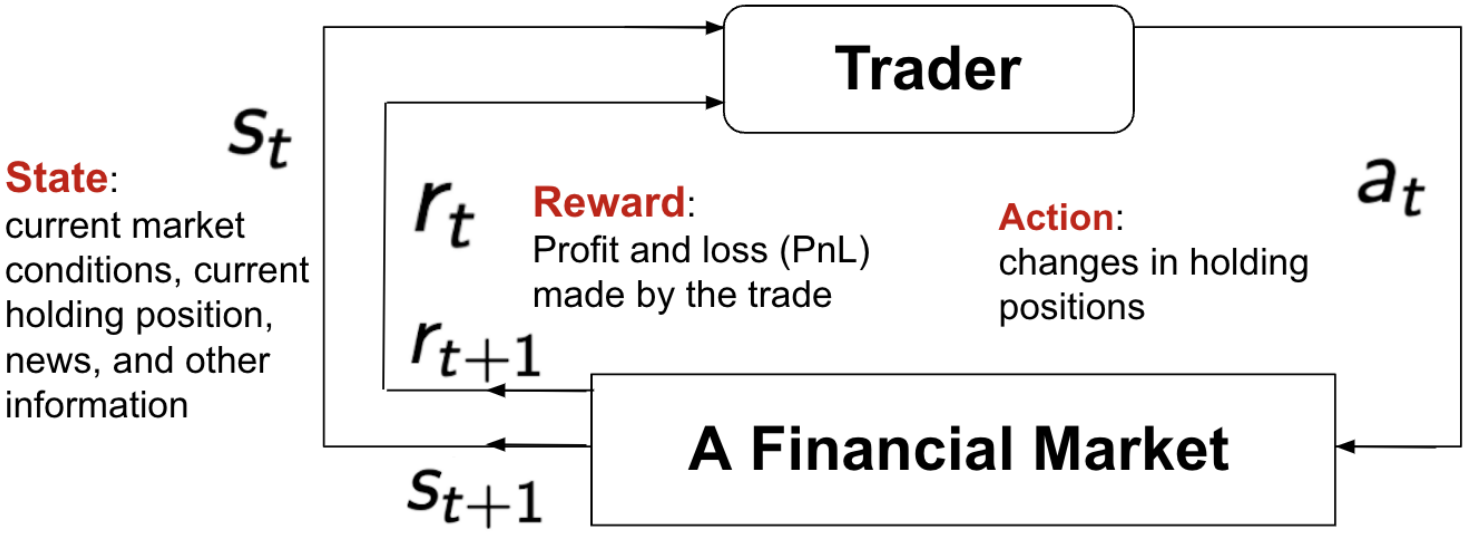
What is Reinforcement Learning?

Reinforcement learning (RL) is a general-purpose framework for decision-making:

- An agent (learner or decision-maker) operates in an environment (possibly unknown to the agent)
- An agent takes actions (a_t) which influences its future state in the environment (s_{t+1})
- Success is measured by a reward signal (r_t) (agent may not know the distribution of the signal)
- **Goal** is to select actions to maximize future rewards



Reinforcement Learning in Finance



Reinforcement learning (RL) in the financial context:

- A trader (or investor) operates in an environment (possibly unknown to the trader)
- A trader takes actions (a_t) which influences its future state (think about the market impact or price impact) in the market (s_{t+1})
- Success is measured by a reward signal (r_t) (such as return or risk-adjusted return)
- **Goal:** is to select actions to maximize future rewards (profit)

Example: Optimal Execution

Problem Description:

- Optimal execution of large (buying and/or selling) positions over a given trading period is an important question for all banks, hedge funds and institutional investors.
- Trade-off between (1) fast trading speed with less profitable execution prices and (2) slow trading speed with the risk of exposure to unknown market fluctuations.

Example: Optimal Execution

Problem Description:

- Optimal execution of large (buying and/or selling) positions over a given trading period is an important question for all banks, hedge funds and institutional investors.
- Trade-off between (1) fast trading speed with less profitable execution prices and (2) slow trading speed with the risk of exposure to unknown market fluctuations.

$\hookrightarrow$ formulate the optimal execution problem in the context of RL:

- **State variable** s: mid-price, current inventory, and other market conditions
- **Action** a: number of shares to liquidate at any timestamp
- **Reward** r: cash inflow, implementation shortfall, Sharpe ratio, return, or PnL

Main Framework for RL: Markov Decision Process

- Markov decision processes (MDP) formally describe an environment for reinforcement learning
- RL is often used to solve MDPs when the transition dynamics and reward function are unknown.

- **Key property (informally):** the **current state** completely characterises the process (no history information is required)
- Agent fully observes the current state before making a decision

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Markov Decision Process

A MDP is specified by a tuple $(\mathcal{S}, s_1, \mathcal{A}, P, R, H)$,

- $\mathcal{S}$ is the set of states (assumed to be finite with size $S = |\mathcal{S}|$)
- $s_1 \in \mathcal{S}$ the starting state
- $\mathcal{A}$ is the set of actions (assumed to be finite with size $A = |\mathcal{A}|$)

Markov Decision Process

A MDP is specified by a tuple $(\mathcal{S}, s_1, \mathcal{A}, P, R, H)$,

- $\mathcal{S}$ is the set of states (assumed to be finite with size $S = |\mathcal{S}|$)
- $s_1 \in \mathcal{S}$ the starting state
- $\mathcal{A}$ is the set of actions (assumed to be finite with size $A = |\mathcal{A}|$)
- The process proceeds in discrete times/rounds $t = 1, 2, ..., H$, starting in the initial state s_1
- In every round t, the agent observes the current state $s_t \in \mathcal{S}$, takes an action $a_t \in \mathcal{A}$, and then observes
  - the next state $s_{t+1} \in \mathcal{S}$
  - a feedback in term of a reward signal $r_t \in \mathbb{R}$

Markov Decision Process

A MDP is specified by a tuple $(\mathcal{S}, s_1, \mathcal{A}, P, R, H)$,

- $\mathcal{S}$ is the set of states (assumed to be finite with size $S = |\mathcal{S}|$)
- $s_1 \in \mathcal{S}$ the starting state
- $\mathcal{A}$ is the set of actions (assumed to be finite with size $A = |\mathcal{A}|$)
- The process proceeds in discrete times/rounds $t = 1, 2, ..., H$, starting in the initial state s_1
- In every round t, the agent observes the current state $s_t \in \mathcal{S}$, takes an action $a_t \in \mathcal{A}$, and then observes
  - the next state $s_{t+1} \in \mathcal{S}$
  - a feedback in term of a reward signal $r_t \in \mathbb{R}$

Example revisited the execution problem:

- **State variable** s: mid-price, current inventory, and other market conditions
- **Action** a: number of shares to liquidate at any timestamp
- **Reward** r: cash inflow, implementation shortfall, Sharpe ratio, return, or PnL

Markov Property

"The future is independent of the past given the present"

Markov Property

"The future is independent of the past given the present"

The Markov property is formally stated as the following two properties:

- The probability of transitioning to a particular state depends only on current state and action, and not on any other aspect of the history. That is,

$$\begin{aligned}&\mathbb{P}(s_{t+1} = s' | \text{history till time t})\\&= \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a) =: P_t(s, a, s')\end{aligned}$$

  Here $\sum_{s'} P_t(s, a, s') = 1$.

- The reward distribution depends only on the current state and action. So, that the expected reward at time t is a function of current state and action. A matrix R specifies these rewards.

$$\begin{aligned}&\mathbb{E}[r_t \,|\, \text{history till time t}]\\&= \mathbb{E}[r_t \,|\, s_t = s, a_t = a] =: R_t(s, a)\end{aligned}$$

Markov Property

Example revisited the execution problem under the Markov assumption:

- The state (e.g., mid-price, current inventory, and other market conditions) at the next timestamp only depends on the current state and the current action (liquidated shares)
- The PnL received in the current period only depends on the current state and current action

$\Longleftarrow$ These are reasonable assumptions to make.

Policy

A policy specifies what action to take at any time step:

- A history dependent policy at time t is a mapping from history till time t to an action
- A Markovian policy is a mapping from state space to action action $\pi : \mathcal{S} \rightarrow \mathcal{A}$
- Due to Markovian property of the MDP, it suffices to consider Markovian policies (in the sense that for any history-dependent policy, same performance can be achieved by a Markovian policy)
- **RL is a method that learns an optimal policy when the MDP (transition probability, reward function) is unknown or partially known.**

Deterministic vs. Randomized Policies

- A **deterministic policy** $\pi : \mathcal{S} \to \mathcal{A}$ is mapping from any given state to an action (often used in Control Theory)
  - Example: long a position (of one unit) if the signal falls below -std; short a position (of one unit) if the signal rises above +std; otherwise do nothing
- A **randomized policy** π: $\mathcal{S} \to \mathcal{P}(\mathcal{A})$ is a mapping from any given state to a distribution over actions (often used in RL)
  - Example: Enter a long position when a signal rises above +std and the position is uniformly sampled from $\{1, 2, 3, 4, 5\}$.
- Following a policy π_t at time t means that if the current state $s_t = s$, the agent
  - takes action $a_t = \pi_t(s)$
  - sample a_t from $\pi_t(s) \in \mathcal{P}(\mathcal{A})$ for randomized policy

Objective of MDP

Goals. The trade-offs between immediate reward vs. future rewards of the sequential decisions and the need for planning ahead is captured by the goal of the MDP.

At a high level, the goal is to maximize expected cumulative reward. Some popular forms are

- (expected) total reward
- (expected) average reward
- (expected) discounted sum of rewards

Finite-horizon MDP

- Here, actions are taken for $t = 1, \cdots, H$ where H is a finite horizon.
- The total (discounted) reward criterion is simply to maximize the expected total (discounted) rewards in an **episode** of length H. [1]
- For discount $0 \leq \gamma \leq 1$, the goal is to maximize:

$$\mathbb{E}\left[\sum_{t=1}^{H} \gamma^{t-1} r_t \middle| \; s_1 \right] \tag{11}$$

  Usually, $\gamma = 1$ when H is small
- Example: option pricing, where time matters

Remark 1: In the RL context, MDP with under a finite time horizon is often referred to as an episodic MDP

Infinite horizon MDP

Expected total discounted reward criteria: the goal is to maximize

$$\mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \,\middle|\, s_1\right] \tag{12}$$

- This is an asymptotic weighted sum of rewards, where with time the weights decrease by a factor of $\gamma < 1$
- This essentially means that the immediate returns are more valuable than those far in the future
- One of the most popular form – theoretically easier to analyze and makes sense for many practical applications
- Example: long-term portfolio optimization

Infinite horizon MDP

Expected total reward criteria: Maximize

$$\lim_{T\to\infty} \mathbb{E}\left[\sum_{t=1}^{T} r_t \,\middle|\, s_1\right] \tag{13}$$

Expected average reward criteria: Maximize

$$\lim_{T\to\infty} \mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T} r_t \,\middle|\, s_1\right] \tag{14}$$

- Intuitively, the performance in a few initial rounds does not matter here, what we are looking for is a good asymptotic performance

Optimal Policies

- Optimal policy is defined as the one that maximizes the gain of the MDP.
- Due to the structure of MDP it is not difficult to show that it is sufficient to consider **Markovian policies**
- Henceforth, we consider only Markovian policies

Optimal Policies

- Optimal policy is defined as the one that maximizes the gain of the MDP.
- Due to the structure of MDP it is not difficult to show that it is sufficient to consider **Markovian policies**
- Henceforth, we consider only Markovian policies

Let us start with a discrete time MDP with an infinite time horizon and discounted reward.

- Value Function
- Q Function

State-Value Function $V^\pi(s)$

The state-value function under policy π is:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s \right]$$

- Expected total reward when starting from state s and following policy π
- Measures how good a state is under a given policy.
- Accounts for both immediate and future rewards.
- Helps compare different states for long-term planning.

Action-Value Function $Q^\pi(s, a)$

Knowing the value of a state isn't enough!

The action-value function measures how good it is to take action a in state s:

$$\begin{aligned} Q^\pi(s,a) &= \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R\left(S_t, A_t\right) \mid S_0 = s, A_0 = a \right] \\ &= R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\pi(s') \end{aligned}$$

Why is $Q^\pi(s, a)$ Important?

- $V^\pi(s)$ tells how good the current state is.

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) Q^\pi(s,a)$$

- $Q^\pi(s, a)$ tells the best action in a given state.

How to compute them efficiently?

Bellman Equations

Recursive Definition:

$$V^{\pi}(s) = \sum_{a \in A} \pi(a|s) \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi}(s') \right]$$

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) \sum_{a'} \pi(a'|s') Q^{\pi}(s',a')$$

Intuition:

- Breaks down the value function into smaller steps.
- Instead of simulating the entire future, we just estimate the next step and recurse.
- Helps compute values efficiently via Dynamic Programming.

Example: Computing the Bellman Equation

- States: s_1, s_2
- Action: a (only one available)
- Rewards: $R(s_1, a) = 5$, $R(s_2, a) = 2$
- Transition Probabilities:
  - From s_1: $P(s_1|s_1, a) = 0.8$, $P(s_2|s_1, a) = 0.2$
  - From s_2: $P(s_2|s_2, a) = 1.0$
- Discount Factor: $\gamma = 0.9$

Example: Computing the Bellman Equation

- States: s_1, s_2
- Action: a (only one available)
- Rewards: $R(s_1, a) = 5$, $R(s_2, a) = 2$
- Transition Probabilities:
  - From s_1: $P(s_1|s_1, a) = 0.8$, $P(s_2|s_1, a) = 0.2$
  - From s_2: $P(s_2|s_2, a) = 1.0$
- Discount Factor: $\gamma = 0.9$

Bellman Equations:

$$V(s_1) = 5 + 0.9\Big[0.8\, V(s_1) + 0.2\, V(s_2)\Big]$$

$$V(s_2) = 2 + 0.9\, V(s_2)$$

Solution:

- $V(s_2) = 2 + 0.9\, V(s_2) \Rightarrow V(s_2) = 20$
- $V(s_1) = 5 + 0.9\Big[0.8\, V(s_1) + 4\Big] = 8.6 + 0.72\, V(s_1)$
- $\Rightarrow V(s_1) \approx 30.71$

Optimal Value Function $V^*(s)$

The optimal value function is:

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \max_{a} Q^*(s, a)$$

where $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$.

Intuition

- $V^*(s)$ represents the best possible value of being in a state.
- $Q^*(s, a)$ represents the maximum expected reward the agent can achieve starting from state s, taking action a, and following the optimal policy thereafter
- Helps us derive the optimal policy.

Optimal Policy π^*

The optimal policy chooses the action that maximizes future rewards:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

Key Takeaways:

- Always select the action with the highest $Q^*(s, a)$.
- Leads to the best possible long-term outcome.

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Two Approaches

RL methods can be classified into:

- **Model-Based RL**: Uses a transition model $P(s'|s, a)$ to simulate experiences and plan actions.
  - maintain an approximate MDP model by estimat- ing the transition probabilities and the reward function
  - - Requires knowledge or estimation of $P(s'|s, a)$, which is often difficult.
  - - Learning an accurate model is challenging.

- **Model-Free RL**: Learns optimal policies purely from experience
  - do not assume knowledge of transition probabilities and reward
  - directly learns state-value or Q function

Model-Free RL

Key Idea: The agent does not know $P(s'|s,a)$. Instead, it learns purely from trial and error.

- $+$ Does not require explicit knowledge of $P(s'|s,a)$.
- $+$ Works well in unknown or complex environments.
- - Less sample efficient; requires more training data.
- - No planning, only learns from past experiences.

Model-Based vs. Model-Free RL

Feature	**Model-Based RL**	**Model-Free RL**
Uses $P(s'\|s, a)$?	Yes	No
Learning Type	Learns from model	Learns from experience
Efficiency	More sample efficient	Less sample efficient
Planning Ability	Can plan future actions	No planning, only reaction
Example Algorithms	DP, MCTS, Dyna-Q	Q-Learning, PPO, DDPG
Best Used For	Simulatable environments	Unknown environments

Optimal Execution

- **State:** (t, X_t, P_t)
- **Action:** Trading rate v_t
- **Reward:** Negative execution cost, e.g.,

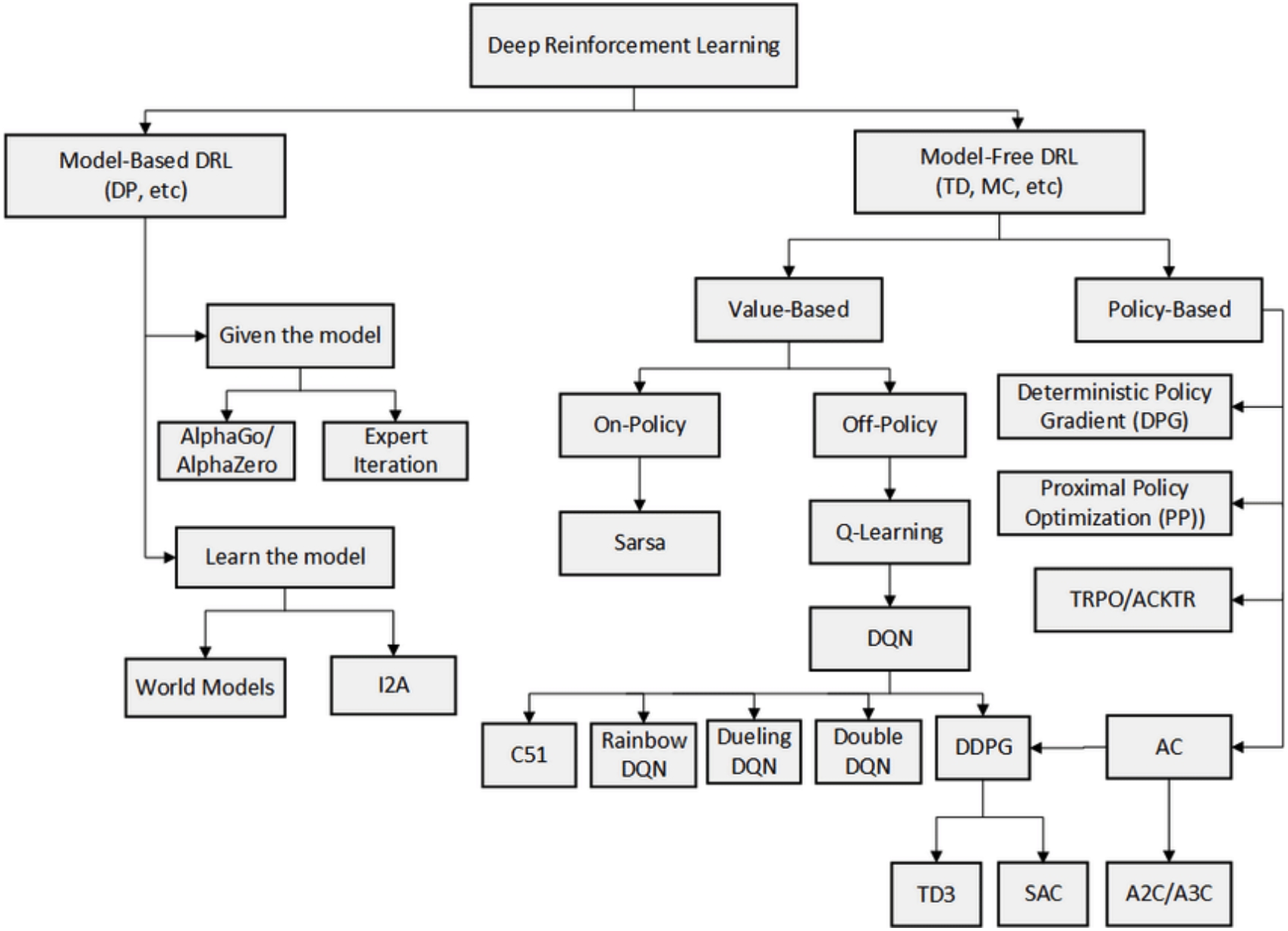
$$R(s,a) = -\left[v_t \cdot \left(P_t - \eta\, v_t\right)\right]$$

  where η is the temporary impact parameter.
- **Transition Dynamics:**
  - Inventory update: $X_{t+1} = X_t - v_t$
  - Price update (permanent impact):
    $P_{t+1} = P_t - \gamma\, v_t + \underbrace{\epsilon_t}_{\text{market uncertainty}}$
- **Model-Based RL:** if the dynamics and noise distribution are known, use dynamic programming to compute $V^*(s)$ and the optimal policy $\pi^*(s)$.
- **Model-Free RL:** Learns directly from experience with updates.

Algorithms

- Value-based methods
- focus on estimating *a value function (usually the Q-function)* and then deriving a policy from it (typically by choosing the action with the highest estimated value).
  - Q-Learning (Off-policy)
  - SARSA (On-policy)
- Policy-based methods
- focus directly on *learning the policy (i.e., the mapping from states to actions)* without explicitly maintaining a value function.
- REINFORCE
- Hybrid Approaches: Actor-Critic

Algorithm List



Off-Policy Methods

Definition:

- Learn about the optimal policy π^* independent of the behavior policy.
- Updates use data that may come from a different (or older) policy.

Example: Q-Learning

$$Q^{(n+1)}(s,a) \leftarrow (1-\beta_n(s,a)) \underbrace{Q^{(n)}(s,a)}_{\text{current estimate}} + \beta_n(s,a) \underbrace{\left[r(s,a) + \gamma \max_{a'} Q^{(n)}\left(s', a'\right) \right]}_{\text{new estimate}}$$

- β is learning rate
- $\max_{a'} Q^{(n)}(s', a')$ is used even if the action taken in s' differs.

Key Benefit:

- Can learn from past experiences or data generated by a different policy.

Q-learning

ALGORITHM 2 *Q*-learning with samples from a policy π

Input: total number of iterations N; the policy π to be evaluated; rule to set learning rate $\beta_n \in (0, 1]$ $(0 \leq n \leq N-1)$
Initialize $Q^{(0)}(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$
Initialize s
for $n = 0, \cdots, N-1$ **do**
  Sample action a according to $\pi(s)$
  Observe r and s' after taking action a
  Update $Q^{(n+1)}$ according to (2.23) with sample (s, a, r, s')
  $s \leftarrow s'$
end for

On-Policy Methods

Definition:

- Learn the value of the current policy π that is used for decision making.
- Updates are based on the actions actually taken by the policy.

Example: SARSA

$$Q^{(n+1)}(s,a) \leftarrow (1-\beta_n(s,a)) \underbrace{Q^{(n)}(s,a)}_{\text{current estimate}} + \beta_n(s,a) \underbrace{\left[r(s,a) + \gamma Q^{(n)}\left(s',a'\right)\right]}_{\text{new estimate}}$$

- The action a' is drawn from the same policy π used in training.

Key Benefit:

- Naturally aligned with the current behavior, which can provide more stable learning.

SARSA

ALGORITHM 3 SARSA: On-policy TD learning

Input: total number of iterations N, the learning rate $\beta \in (0,1)^1$, and small parameter $\varepsilon > 0$.
Initialize $Q^{(0)}(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$
Initialize $s \in \mathcal{S}$
Initialize a: Choose a when in s using a policy derived from $Q^{(0)}$ (e.g, ε-greedy)
for $n = 0, 1, \cdots, N-1$ **do**
  Take action a, observe reward r and the next step s'
  Choose a' when in s' using a policy derived from $Q^{(n)}$ (e.g., ε-greedy)

$$Q^{(n+1)}(s,a) \leftarrow (1-\beta) \underbrace{Q^{(n)}(s,a)}_{\text{current estimate}} + \beta \underbrace{(r + \gamma Q^{(n)}(s',a'))}_{\text{new estimate}}$$

  $s \leftarrow s'$, $a \leftarrow a'$
end for

The policy derived from $Q^{(n)}$ using ε-greedy to choose action a when in state s :

$$\begin{cases} \text{select from } \arg\max_{a'} Q^{(n)}(s,a') & \text{with probability } 1-\varepsilon, \\ \text{uniformly sample from } \mathcal{A} & \text{with probability } \varepsilon. \end{cases}$$

Q-learning uses the maximum value of $Q^{(n)}(s,a)$ over all possible actions for the next step

REINFORCE

- **Monte Carlo policy gradient** method.
- It directly optimizes a parameterized policy $\pi_\theta(a|s)$ to maximize the expected return.
- The goal is to maximize:

$$J(\theta) = \mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

where γ is the discount factor.

Pros and Cons:

- $+$ Simple and intuitive to implement.
- $+$ Directly optimizes the policy.
- - High variance in the gradient estimates.
- - Sample inefficiency requires many episodes to converge.

Policy Gradient Theorem (Sutton et al., 2000)

- The gradient of the expected return is given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a|s)\, G_t\right]$$

- Here, G_t is the return (cumulative discounted reward) from time t, i.e. $\sum_{s=t+1}^{M} \gamma^{s-t-1} r_s$.
- This "score function" $\nabla_\theta \log \pi_\theta(a|s)$ tells us how to adjust θ to increase the likelihood of taking actions that yield high rewards.

The REINFORCE Algorithm

Steps:

1. **Collect an Episode:** Run the current policy π_θ to generate a trajectory:
$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_T)$$
2. **Compute Returns:** For each time step t, compute:
$$G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$
3. **Update Policy:** For each time step t, update parameters:
$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t)\, G_t$$
where α is the learning rate.

Actor–Critic Methods

- The approach consists of two main components:
  - **Actor:** Represents the policy $\pi_\theta(a|s)$, which selects actions.
  - **Critic:** Estimates the value function (e.g., $V^\pi(s)$ or $Q^\pi(s,a)$) to evaluate actions.

Actor:

- Parameterized policy $\pi_\theta(a|s)$
- Directly maps states to actions (or action probabilities)

Critic:

- Estimates the value function $V^\pi(s)$ or $Q^\pi(s,a)$
- Provides feedback on the expected long-term reward

Watch Actor-Critic

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Background and Motivation

- **Q-Learning Recap:**
  - Learns the optimal Q-function $Q^*(s, a)$ via:

$$Q^*(s, a) \leftarrow R(s, a) + \gamma \max_{a'} Q^*(s', a')$$

  - Traditional tabular methods become infeasible in high-dimensional state spaces.
- **Need for Function Approximation:**
  - Deep neural networks can approximate the Q-function.

What is DQN?

- **Deep Q-Networks (DQN)** use a deep neural network to approximate:
$$Q(s, a; \theta) \approx Q^*(s, a)$$
- **Inputs:** Raw, high-dimensional state (e.g., game screen images).
- **Outputs:** Q-values for all possible actions.

DQN Algorithm Outline

1. Initialize Q-network with weights θ and target network with $\theta^- = \theta$.
2. For each episode:
   - Start from initial state s.
   - For each time step:
     - Select action a using an ϵ-greedy strategy.
     - Execute a, observe r and next state s'.
     - Store (s, a, r, s') in replay memory.
     - Sample a mini-batch from replay memory.
     - Compute target:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

     - Update Q-network by minimizing the loss:

$$L(\theta) = \mathbb{E}\left[(y - Q(s, a; \theta))^2\right]$$

   - Periodically update target network $\theta^- \leftarrow \theta$.

Why DQN is Important

- **Handles High-Dimensional Inputs:**
  - Learns directly from raw pixel data (e.g., Atari games).
- **Stability Enhancements:**
  - Experience replay and target networks reduce instability issues in learning.
- **Foundation for Extensions:**
  - Inspired numerous extensions (Double DQN, Dueling DQN, etc.) to improve performance.

Google DeepMind's Deep Q-learning playing Atari Breakout

Optimal Execution

Basics about Reinforcement Learning

Markov Decision Process

Reinforcement Learning

Deep RL

RL in Finance

Optimal Execution

- **Early Approaches:**
  - Nevmyvaka et al. [2006]: Pioneered RL-based execution strategies.
- **Deep RL Breakthroughs:**
  - Ning et al. [2021]: Applied Deep Q-Networks (DQN) to handle high-dimensional market data.
- **Multi-Agent Simulations:**
  - Trading agent is situated in a simulated market alongside other agents (representing other traders or liquidity providers) to capture the interactive nature of order execution.
- **Institutional Deployment:**
  - RBC's Aiden: Uses deep RL for optimal execution.

Other Applications

- Beyond optimal execution, RL is applied in multiple areas of finance.
- Key domains include Hambly et al. [2023]:
  - Portfolio Management and Algorithmic Trading
  - Market Making and Risk Management
  - Option Pricing and Optimal Exercise Strategies
  - Credit Scoring and Fraud Detection

Portfolio Management & Algorithmic Trading

- **Portfolio Management:**
  - RL agents dynamically adjust asset allocations to optimize returns and control risk.
  - Strategies include dynamic rebalancing and risk-adjusted asset selection.
- **Algorithmic Trading:**
  - RL is used to develop trading strategies that determine entry and exit timing.
  - Applications span high-frequency trading, statistical arbitrage, and exploiting market inefficiencies.

Market Making & Risk Management

- **Market Making:**
  - RL agents set bid and ask quotes to provide liquidity while managing inventory risk.
  - The goal is to optimize the balance between profit from spreads and the cost of holding positions.
- **Risk Management and Hedging:**
  - RL techniques design dynamic hedging strategies.
  - The agent learns to adjust hedges based on market volatility and risk exposures.

Option Pricing, Exercise Strategies, & Fraud Detection

- **Option Pricing & Optimal Exercise:**
  - RL is applied to price American options and determine optimal exercise policies.
  - It handles the optimal stopping problem in a dynamic setting.
- **Credit Scoring & Fraud Detection:**
  - RL methods optimize decision policies for loan approvals and risk assessments.
  - They are also used to detect and adapt to emerging patterns in fraudulent activities.

Q&A

- Questions or thoughts about today's lecture?

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3:5–40, 2001.

Ben Hambly, Renyuan Xu, and Huining Yang. Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3):437–503, 2023.

Yuriy Nevmyvaka, Yi Feng, and Michael Kearns. Reinforcement learning for optimized trade execution. In *Proceedings of the 23rd international conference on Machine learning*, pages 673–680, 2006.

Brian Ning, Franco Ho Ting Lin, and Sebastian Jaimungal. Double deep q-learning for optimal execution. *Applied Mathematical Finance*, 28(4):361–380, 2021.